

DevOps Assignments Study Guide — Git, Jenkins, Docker, Kubernetes, Ansible & Terraform

Goal: be able to explain each assignment briefly and confidently in conversation, not recite commands from memory.

Meeting-Ready Answer (Kubernetes & Terraform Focus)

Since the team already uses Kubernetes and Terraform, lead with these two and keep the rest brief. This is the version to actually rehearse.

“I’ve been working through a hands-on DevOps course — Git, Jenkins, Docker, Kubernetes, Ansible, and Terraform — so I’ve got practical exposure across the pipeline, not just theory.

The two I’ve spent the most time on are Kubernetes and Terraform, since I know that’s what your team actually uses. On Kubernetes, I’ve worked with deployments — which manage **Pods**, the smallest deployable unit in Kubernetes, usually one container each — and I’ve practiced scaling them up and down, and exposing them three different ways: NodePort, ClusterIP for internal-only access, and Ingress, which I understand is closer to how production traffic is usually routed.

On Terraform, I’ve gone from provisioning single resources, to building a full custom VPC with proper subnets and routing, to multi-region deployment and basic server bootstrapping.

Alongside that, I’ve also covered Git, Jenkins, Docker, and Ansible, so I understand how those fit around Kubernetes and Terraform in a typical pipeline.”

Then pivot with a genuine question (this matters more than reciting more detail): > “Since your team already uses Terraform and Kubernetes, I’d be curious how your real setup differs from a course environment — do you manage Terraform state remotely, use modules heavily? And on the Kubernetes side, are you managing pods directly or is that mostly abstracted away through ArgoCD?”

One-sentence pod explainer, in case he asks directly: > “A pod is the smallest unit in Kubernetes — usually just one container — and a deployment’s job is to keep a set number of them running and healthy, replacing any that crash automatically.”

Delivery note: don’t try to recite all six tool summaries evenly — that reads as reciting notes rather than talking naturally. Know Kubernetes and Terraform solidly, keep the rest as “I’ve touched on this too,” and let the question at the end do the work of showing genuine curiosity about their actual setup.

Meeting Script — Full Walkthrough (Start to Finish)

This is the consolidated version to actually rehearse. No code — just the narrative, in order.

1. Opening — What you’ve been doing

“I’ve been doing a hands-on DevOps course over the past while — Git, Jenkins, Docker, Kubernetes, Ansible, and Terraform — working through practical exercises rather than just theory.”

2. Why DevOps interests you

“I run some of my own projects too, including one where I built an automated pipeline using GitHub Actions — pulling data, generating content, and delivering it on a schedule with no manual input. That’s what got me hooked on the automation and infrastructure side, which is what pulled me toward DevOps specifically.”

3. Be upfront about your level

“I’ll be honest though — I’m still early in all of this. I’ve typed the commands and seen things actually work, which has helped it stick, but I don’t have it all memorized, and there’s a lot I still need to build real experience with.”

4. The tools, briefly — what each one is

- **Git** — version control. Tracks changes to code over time, lets you branch off to work on things separately, and merge it all back together.
- **Jenkins** — automation server. Runs your pipeline automatically whenever code changes — building, testing, and deploying without someone doing it by hand.
- **Docker** — packages an app and everything it needs into a portable container, so it runs the same way anywhere.
- **Ansible** — automates installing and configuring software across many machines at once, instead of doing it manually on each one.
- **Kubernetes** — manages containers across a cluster of machines. You tell it “keep 3 copies of this running,” and it handles the rest.
- **Terraform** — infrastructure as code. Instead of manually setting up servers and networks in a console, you write what you want in a config file, and it builds it for you.

5. Go deeper on Kubernetes and Terraform specifically

“Since I know your team already uses Kubernetes and Terraform, that’s where I’ve focused most.”

Kubernetes — a fuller explanation: > “Kubernetes is a system for running and managing containers across a cluster of machines, rather than on just one server. The core idea is ‘desired state’ — you tell it something like ‘I want 3 copies of this app running at all times,’ and Kubernetes constantly checks the real state of the cluster against that and corrects any drift. If a container crashes, or even if a whole machine goes down, it automatically creates a replacement elsewhere in the cluster to bring things back to that target — that self-healing behaviour is the main thing that sets it apart from just running containers manually. > > The smallest unit Kubernetes manages is called a **pod** — usually just one container, sometimes a couple tightly coupled ones. A **deployment** is the object that actually manages a group of pods and keeps that replica count correct. And because pods come and go and their addresses change, Kubernetes uses something called a **service** to give a stable, unchanging way to reach whichever pods are currently healthy — so nothing else in the system needs to know or care that pods are being replaced underneath it. > > I’ve practiced deploying an app, scaling the replica count up and down, and exposing it a few different ways — some only reachable from inside the cluster, and others properly reachable from outside, which I understand is closer to how a production setup like yours would actually route real traffic.”

Terraform — a fuller explanation: > “Terraform is infrastructure as code — instead of manually clicking through AWS or Azure to create servers, networks, and other resources, you describe what you want in a config file, and Terraform builds it for you. If you update the file and run it again, it works out the difference and updates the real infrastructure to match, rather than you tracking changes by hand. > > The workflow has a few key steps: **plan**, which shows you exactly what will be created, changed, or destroyed *before* anything actually happens — so you’re never applying changes blind — and **apply**, which actually makes it happen. That preview step is a big part of why it’s trusted for real infrastructure, since nothing happens silently. > > I’ve gone from provisioning a single server, up to building a proper custom network — a VPC with its own subnet and routing — and deploying infrastructure across more than one AWS region from a single setup. It’s given me a decent sense of how infrastructure gets defined and managed as code, rather than built manually and hoping it’s remembered correctly next time.”

6. Frame your internal move honestly

“I’m currently in IT support here, and I’ve been using that as a starting point to move toward DevOps — it’s given me a decent foundation, since I’m already used to troubleshooting systems

and understanding infrastructure, just from a different angle.”

7. Pivot to a genuine question

“Since your team already uses Terraform and Kubernetes, I’d be curious how your real setup differs from a course environment — do you manage Terraform state remotely, use modules heavily? And on the Kubernetes side, are you working with pods directly day to day, or is a lot of that abstracted away through ArgoCD?”

8. If asked “where do you see yourself in a few years”

“Ideally, comfortable with the core tools day to day, and probably starting to specialize a bit more — maybe cloud infrastructure or automation. Right now my focus is just building a strong foundation, and being internal already, I’m hoping that gives me a bit of a head start.”

Quick Tool Definitions (from the manager’s outline)

Short, one-breath explanations to show baseline understanding of the specific tools mentioned in Nicolas’s outline — not meant to sound like expertise, just enough to hold a conversation.

Kubernetes > “It’s the system that runs and manages containers across a cluster of machines — it keeps a set number of copies of an app running, and automatically replaces any that crash, so nothing goes down unnoticed.”

Terraform > “It’s infrastructure as code — instead of manually setting up servers and networks in a console, you write it in a config file, and Terraform builds it for you. Makes environments repeatable and version-controlled.”

Confluent Cloud > “It’s a managed version of Kafka — Kafka lets different applications pass data to each other in real time without being directly connected, and Confluent Cloud means you don’t have to run the Kafka servers yourself.”

Assignment 1 — Git Basics

In one sentence: Set up Git, initialize a repo, and do a first commit.

What I did: Configured Git with my name/email, created a folder with a few files, staged specific files, committed them, and pushed to GitHub.

Commands:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"

mkdir my-project && cd my-project
git init

git add Code.txt Output.txt          # only staged specific files, not Log.txt
git commit -m "Initial commit"

git remote add origin <repo-url>
git push -u origin master
```

Concept to remember: Git has 3 states — **working directory** (edited) → **staging area** (`git add`) → **repository** (`git commit`). Staging lets you choose exactly what goes into each commit, rather than saving everything at once.

Assignment 2 — Branching & Stashing

In one sentence: Used branches for parallel work, and `git stash` to safely switch branches without losing uncommitted changes.

What I did: Created branches, made an uncommitted change on one branch, stashed it, switched branches to do other work, then came back and restored the stashed change.

Commands:

```
git branch develop
git branch feature1
git branch feature2

git checkout develop
touch develop.txt           # left uncommitted on purpose

git stash -u                # -u = include untracked files too
git checkout feature1

touch new.txt
git add new.txt
git commit -m "Add new.txt on feature1"

git checkout develop
git stash pop               # brings the stashed file back
git add develop.txt
git commit -m "Add develop.txt after unstashing"
```

Concept to remember: `git stash` = a shelf for unfinished work. `stash pop` restores it and clears the shelf; `stash apply` restores it but keeps a copy. By default `stash` ignores untracked (never-added) files — that's why `-u` was needed.

Assignment 3 — Remote Branches & Deletion

In one sentence: Pushed multiple branches to GitHub, then deleted one branch both locally and remotely.

What I did: Created branches with different files, pushed them all to GitHub, then cleaned up by deleting a branch that was no longer needed.

Commands:

```
git branch develop
git branch f1
git branch f2

git push -u origin master
git push -u origin develop
git push -u origin f1
git push -u origin f2
```

```
git checkout master           # must leave a branch before deleting it
git branch -d f2              # safe delete (only if merged)
git branch -D f2              # force delete (if not merged)

git push origin --delete f2    # delete it on GitHub too
```

Concept to remember: Deleting a branch locally and deleting it on the remote are **two separate steps** — Git never syncs deletions automatically in either direction.

Assignment 4 — Merging & Keeping Branches in Sync

In one sentence: Merged feature branches into master, then simulated master changing independently and pushed that update back out to the other branches.

What I did: Merged two feature branches into master, edited master from a separate branch, then propagated that change into every other active branch so nothing was left out of date.

Commands:

```
git checkout master
git merge public1
git merge public2

git checkout private
echo "updated content" >> master.txt
git add master.txt
git commit -m "Update master.txt on private branch"

git checkout public1
git merge private
git checkout public2
git merge private

git checkout master
git merge private

git checkout private
git merge master           # bring everything full circle
```

Concept to remember: In real teams, a shared branch gets updated constantly, and that update has to be merged *into* every active feature branch — otherwise people drift onto stale code and hit painful conflicts later.

Assignment 5 — Git Flow Workflow

In one sentence: Set up the full Git Flow branching model and simulated a feature going live, plus an emergency hotfix.

What I did: Created the standard Git Flow branch structure, pushed a feature through the proper chain into production, and simulated an urgent hotfix that bypasses the normal process.

The branch structure (explain this concept first — it matters more than the commands): | Branch | Purpose | |—| | **master** | Always reflects live/production code | | **develop** | Main integration

branch where features come together | | **feature/x** | One branch per new feature, branched off **develop** | | **release/x** | Prep branch for a new release, branched off **develop** | | **hotfix/x** | Urgent production fix, branched directly off **master** |

Commands:

```
git checkout -b develop master
git checkout -b feature/login develop

touch login.txt
git add login.txt
git commit -m "Add login feature"

git checkout develop
git merge feature/login

git checkout -b release/1.0 develop

git checkout master
git merge release/1.0
git tag -a v1.0 -m "Release version 1.0"

git checkout develop
git merge release/1.0

# Hotfix - urgent, skips the normal process
git checkout -b hotfix/urgent-fix master
touch urgent.txt
git add urgent.txt
git commit -m "Urgent hotfix"

git checkout master
git merge hotfix/urgent-fix
git tag -a v1.0.1 -m "Hotfix release 1.0.1"

git checkout develop
git merge hotfix/urgent-fix # so develop doesn't lose the fix
```

Concept to remember: Git Flow separates “code being built” (develop, feature) from “code that’s live” (master), with release branches as a controlled gate between them. Hotfix branches are the one shortcut — allowed to go straight to master when production is broken. This is very close to how your team’s ArgoCD/GitOps setup likely works: Git branches representing controlled states that flow into environments.

Jenkins Assignments 1–3

Assignment 1 — Git-Triggered Pipeline

In one sentence: Set up a Jenkins pipeline that automatically runs when code is pushed to the **develop** branch, and pulls the latest code into a folder.

What I did: Configured a pipeline job with a Git trigger, so a push to **develop** automatically kicks off a build that checks out the latest code — no manual step needed.

Config approach (GUI + Jenkinsfile, not really “commands” like Git): - Install the Git plugin - Create a new Pipeline job - Enable a trigger — either a **webhook** (GitHub notifies Jenkins instantly on push) or **Poll SCM** (Jenkins checks on a schedule) - Pipeline script checks out the **develop** branch into the workspace

Concept to remember: Webhook vs polling — a webhook means GitHub actively tells Jenkins “something changed, go now” (instant, efficient). Polling means Jenkins checks on a timer (simpler, but slower and wastes resources). Real production pipelines almost always use webhooks.

Assignment 2 — Master/Agent Nodes + Environment-Based Deployment

In one sentence: Added extra Jenkins nodes (agents), then created two jobs that deploy to different environments depending on which branch was pushed to.

What I did: Registered a test node and a prod node under Jenkins, then built a “push to test” job that deploys to the test server, and a separate “push to prod” job that deploys to the prod server — each triggered by its matching branch.

Config approach: - **Manage Jenkins → Nodes → New Node** — add **test-node** and **prod-node** (Jenkins’ master/agent architecture: the master coordinates, agents do the actual work) - Build two pipeline jobs, each with **agent { label 'test-node' }** or **agent { label 'prod-node' }** to pin it to the right machine - Each job checks out its matching branch (**test** or **master**) and copies the files to that environment’s folder

Concept to remember: This is a simplified form of **environment promotion** — code moves from test to production in a controlled, branch-driven way instead of someone manually copying files around.

Assignment 3 — Chained Pipeline with a Safety Gate

In one sentence: Chained the test and prod jobs together, so prod only runs automatically if test succeeds first.

What I did: Built on Assignment 2 by linking the jobs — a push to **develop** triggers the test job, and only if that job passes does it automatically trigger the prod job. Nothing reaches production unless the previous stage succeeded.

Config approach: - Simplest way: on the prod job, under Build Triggers, use **“Build after other projects are built”** → watch the test job → condition set to **“only if stable”** - More realistic way: a single pipeline script with sequential stages (**Test** then **Prod**), where the **Prod** stage only runs if the **Test** stage succeeded

Concept to remember: the difference between **independent jobs** (Assignment 2) and a **pipeline with a gate** (this one). That “only proceed if the last stage succeeded” pattern is exactly what stops broken code reaching production automatically — and it maps directly onto how Jenkins would hand off to ArgoCD in your team’s real setup: Jenkins gates code through test, then GitOps tools handle getting the approved result into the cluster.

Docker Assignments 1–5

Assignment 1 — First Container + Web Server

In one sentence: Pulled an Ubuntu image, ran it as a container with a mapped port, installed Apache inside it, and confirmed it was reachable in the browser.

What I did: Proved a container can behave like a real, network-reachable server — the basic “hello world” of Docker.

Concept to remember: `-p 80:80` maps a container’s internal port to your machine — without it, Apache would be running inside the container but invisible to your browser. An **image** is a static template; a **container** is a running instance of it.

Assignment 2 — Committing a Container to a Custom Image

In one sentence: Saved my customized container (Ubuntu + Apache) as a new reusable image using `docker commit`, then proved a fresh container from that image already had Apache built in.

What I did: Turned a one-off manual setup into a reusable template, instead of repeating the install every time.

Concept to remember: `docker commit` freezes a running container’s current state into a new image — quick, but considered a bit of an anti-pattern since it’s not repeatable or reviewable. The proper version of this is a **Dockerfile** (Assignment 4).

Assignment 3 — Pushing to Docker Hub

In one sentence: Pushed my custom image to Docker Hub, then pulled it down on a separate machine and ran it there successfully.

What I did: Proved Docker’s core promise — “build once, run anywhere” — by showing the image behaves identically on a different machine with zero extra setup.

Concept to remember: Docker Hub is a **registry** — same idea as GitHub, but for images. `docker push` uploads, `docker pull` downloads. This is how teams guarantee Dev, Test, and Prod are all running the *exact same* environment.

Assignment 4 — Writing a Dockerfile

In one sentence: Replaced manual setup with a Dockerfile — a script that builds the image automatically and starts Apache the moment the container runs.

What I did: Wrote the same Ubuntu + Apache setup as code, so it’s version-controlled and rebuildable by anyone, instead of a manual snapshot.

Concept to remember: Each line in a Dockerfile is a cached **layer**, which is why rebuilds are fast. Also — CMD needs to keep the main process running in the *foreground*, or the container will exit immediately after starting (this is why `apache2ctl -D FOREGROUND` is used instead of the normal `service apache2 start`).

Assignment 5 — Custom Content via Dockerfile

In one sentence: Extended the Dockerfile to copy in my own HTML file, replacing Apache’s default page with custom content.

What I did: Took the container from “generic demo” to actually serving something specific — the realistic use case of packaging an application inside an image.

Concept to remember: COPY in a Dockerfile pulls a file from your local machine into the image at build time. /var/www/html/ is Apache's web root, so overwriting `index.html` there replaces the default page with your own.

Kubernetes Assignments 1–5

Assignment 1 — Cluster Setup + First Deployment

In one sentence: Stood up a 3-node Kubernetes cluster and deployed NGINX with 3 replicas.

What I did: Proved Kubernetes can manage an app across multiple nodes rather than it just running on one machine, with built-in redundancy.

Concept to remember: A **deployment** declares *desired state* — “always keep 3 healthy copies running.” Kubernetes constantly checks reality against that and auto-replaces anything that dies. That self-healing behavior is the core value over just running containers manually with Docker.

Assignment 2 — Exposing a Deployment with a Service

In one sentence: Exposed the NGINX deployment externally using a NodePort service, and confirmed it in the browser.

What I did: Solved the same problem Docker's `-p` port mapping solves, but at the cluster level — making a deployment actually reachable from outside.

Concept to remember: Pod IPs constantly change as pods are created/destroyed — a **Service** gives a stable endpoint that load-balances across whichever pods are currently healthy. Three main types: **ClusterIP** (internal only, default), **NodePort** (opens a port on every node), **LoadBalancer** (provisions a real external load balancer — the typical production choice on cloud providers).

Assignment 3 — Scaling a Deployment

In one sentence: Scaled the same deployment from 3 replicas to 5 with a single command, with no downtime and no changes to the service.

What I did: Demonstrated that scaling is just updating desired state — Kubernetes handles the rest, and the existing service automatically starts routing to the new pods too.

Concept to remember: This same mechanism is what powers **autoscaling** — Kubernetes can watch CPU/memory and adjust replica count automatically (Horizontal Pod Autoscaler), rather than someone running the `scale` command by hand.

Assignment 4 — Switching Service Type to ClusterIP

In one sentence: Changed the service from NodePort back to ClusterIP, so it's only reachable from inside the cluster.

What I did: Practiced the opposite lesson from Assignment 2 — deciding what *shouldn't* be exposed externally, like an internal API or database.

Concept to remember: This is about **exposure boundaries** — only the parts of a system that genuinely need public traffic (a frontend, an API gateway) should ever be NodePort/LoadBalancer/Ingress. Everything else should stay ClusterIP. Getting this wrong is a real security concern, not just a technical detail.

Assignment 5 — Ingress (Bringing It Together)

In one sentence: Redeployed NGINX behind a ClusterIP service, then used an Ingress to expose it properly instead of NodePort.

What I did: Combined everything from the earlier assignments into the pattern real production clusters actually use — internal by default, exposed deliberately through a controlled routing layer.

Concept to remember: An **Ingress controller** is the actual traffic router (often NGINX itself); an **Ingress resource** is just the rules telling it how to route traffic by hostname/path. One Ingress can route to many services based on the URL — a big step up from opening a separate NodePort per service.

Ansible Assignments 1–5

Assignment 1 — Cluster Setup + First Playbooks

In one sentence: Set up an Ansible control node managing 3 target machines, then wrote playbooks to install Java on one and MySQL on another.

What I did: Automated software installs across multiple machines instead of manually SSHing into each one.

Concept to remember: Ansible is **agentless** — it just connects over SSH, no special software needed on target machines. A **playbook** describes desired state in YAML, and Ansible is **idempotent** — re-running it does nothing if the state already matches, so it's always safe to re-run.

Assignment 2 — Running Custom Scripts Across Hosts

In one sentence: Wrote a shell script and used Ansible to run it on every host in the inventory with a single command.

What I did: Showed Ansible isn't limited to installing packages — it can execute arbitrary scripts across the whole fleet at once.

Concept to remember: The **script** module copies and executes a script remotely, but unlike the package modules from Assignment 1, it has **no idempotency** — it just runs every time, regardless of prior state. **all** is a built-in inventory group covering every host, regardless of which specific group they belong to.

Assignment 3 — Roles

In one sentence: Organized the work into two Ansible roles — one installing Apache on slave1, one installing NGINX on slave2.

What I did: Moved from one flat playbook to a reusable, standardized structure — the way real Ansible projects are actually organized.

Concept to remember: A **role** is a self-contained, reusable package of automation (tasks, files, variables, templates) in a standard folder layout. Comparable to a reusable Terraform module or a Dockerfile — build it once, reuse it anywhere.

Assignment 4 — Deploying Custom Files via a Role

In one sentence: Extended the NGINX role to deploy a custom index.html, replacing the default page — only on the host with NGINX installed.

What I did: Used the role's `files/` folder plus a `copy` task to push actual content through the automation, not just install generic software.

Concept to remember: Because the task lives inside `nginx_role`, and that role is only ever assigned to `slave2`, the custom page only ever lands there — no extra conditional logic needed. Same underlying idea as Docker Assignment 5, just applied to already-running machines instead of at image build time.

Assignment 5 — Inventory Groups + Environment-Based Roles

In one sentence: Scaled to a 5-node cluster, grouped hosts as test/prod in the inventory, and applied different roles to each group.

What I did: Installed Java on the test group and MySQL on the prod group, targeting groups instead of individual hostnames — the realistic way infrastructure is organized.

Concept to remember: Targeting **groups** (`hosts: test`) instead of individual hosts (`hosts: slave1`) is how real infrastructure scales — add a new server's IP under the right group, and every existing role/playbook automatically applies to it. Same underlying job as Jenkins branches (Assignment 2/3 in Jenkins) — different environments getting the right treatment automatically, just at the configuration layer instead of the deployment layer.

Terraform Assignments 1–5

Assignment 1 — First AWS Resource

In one sentence: Created a single EC2 instance in AWS's Ohio region using Terraform, in the default subnet.

What I did: Proved infrastructure can be defined as code and provisioned automatically instead of clicking through the AWS console.

Concept to remember: `terraform plan` shows exactly what will be created/changed/destroyed *before* anything happens — nothing applies silently, which is a big part of why Terraform is trusted for production infrastructure.

Assignment 2 — Attaching an Elastic IP

In one sentence: Destroyed the previous instance, created a new one, and attached an Elastic IP — a fixed public IP that stays constant even if the instance restarts.

What I did: Solved the real problem that a normal EC2 public IP changes every time the instance is stopped/started.

Concept to remember: Referencing one resource inside another (`instance = aws_instance.my_ec2.id`) is how Terraform automatically works out creation order — it knows the EC2 instance must exist before the Elastic IP can attach to it.

Assignment 3 — Multi-Region Deployment

In one sentence: Deployed two EC2 instances simultaneously in different AWS regions (Ohio and N. Virginia) from a single Terraform config.

What I did: Used provider aliasing to manage multi-region infrastructure from one codebase — a real pattern for disaster recovery or serving users closer to their region.

Concept to remember: One `provider` block normally = one region. The `alias` attribute lets you define multiple provider configs of the same type, and each resource picks which one to use. AMI IDs are also **region-specific** — the same AMI doesn't exist across regions.

Assignment 4 — Custom VPC + Networking

In one sentence: Built a custom VPC from scratch — subnet, internet gateway, route table — and deployed an EC2 instance inside it, instead of relying on AWS's default networking.

What I did: Moved from “spin up one server” to genuine infrastructure design — the way real production environments are actually built.

Concept to remember: A public-facing instance needs 4 pieces working together: **VPC** (network boundary), **subnet** (a slice of it), **internet gateway** (the door to the public internet), and **route table** (instructions telling traffic to use that door). Miss any one and the instance is unreachable even with a public IP.

Assignment 5 — Provisioning + Local Output

In one sentence: Created an EC2 instance that installs Apache automatically on boot via a startup script, then wrote its public IP to a local file.

What I did: Combined infrastructure provisioning with basic bootstrapping and local output — bridging Terraform into the kind of setup work Ansible/Docker handle.

Concept to remember: `user_data` runs a script automatically the first time an instance boots — good for simple setup, though anything more complex would typically hand off to Ansible instead. `local_file` shows Terraform resources aren't limited to cloud infrastructure — it can manage local files too.

How to Talk About This Briefly in the Meeting

If asked “what have you been working on,” a short version that covers everything:

“I’ve been working through Git, Jenkins, Docker, Kubernetes, Ansible, and Terraform assignments. On Git, I covered the full workflow — commits, branching, stashing, remote branches, merging strategies, and Git Flow with a hotfix. On Jenkins, I moved from a basic Git-triggered pipeline, to agent nodes and branch-based deployment, up to a chained pipeline with a safety

gate. On Docker, I went from a basic container, to custom images, Docker Hub, and a proper Dockerfile serving my own content. On Kubernetes, I stood up a cluster, deployed and scaled an app, and worked through NodePort, ClusterIP, and Ingress. On Ansible, I went from basic playbooks, to running scripts across a fleet, to reusable roles, to grouping hosts by environment. And on Terraform, I went from provisioning a single EC2 instance, to a stable Elastic IP, to multi-region deployment, to building a proper custom VPC, and finally to bootstrapping a server automatically and capturing its output locally. It's given me a solid practical foundation across the full DevOps toolchain — source control, CI/CD, containerization, orchestration, configuration management, and infrastructure as code.”